Lab-16.1(AIAC)

2303A51494

B27

Task1: Student Information System Schema

PROMPT:

generate SQL create table statement for a student information system that includes tables students, courses and enrollments also define primary keys,foreign keys and relationships between the tables

generate sql queries to insert sample data into the students, courses and enrollments tables

generate sql query to insert a new student record

generate sql query to fetch all courses enrolled by a student

CODE:

lab16.sql

```sql
-- generate  SQL create table statement for a student information system that includes tables students, courses and enrollments
--also define primary keys,foreign keys and relationships between the tables
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    credits INT NOT NULL
);

CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

--generate sql queries to insert sample data into the students, courses and enrollments tables
INSERT INTO students (student_id, first_name, last_name, email) VALUES
(1, 'John', 'Doe', 'john.doe@example.com'),
(2, 'Jane', 'Smith', 'jane.smith@example.com');

INSERT INTO courses (course_id, course_name, credits) VALUES
(101, 'Introduction to Computer Science', 3),
(102, 'Calculus I', 4);

INSERT INTO enrollments (student_id, course_id) VALUES
(1, 101),
(1, 102),
(2, 101);
```

```sql
(2, 'Jane', 'Smith', 'jane.smith@example.com');

INSERT INTO courses (course_id, course_name, credits) VALUES
(101, 'Introduction to Computer Science', 3),
(102, 'Calculus I', 4);

INSERT INTO enrollments (student_id, course_id) VALUES
(1, 101),
(1, 102),
(2, 101);

--generate sql query to insert a new student record
INSERT INTO students (student_id, first_name, last_name, email) VALUES
(3, 'Alice', 'Johnson', 'alice.johnson@example.com');

--generate sql query to fetch all courses enrolled by a student
SELECT c.course_name
FROM courses c
JOIN enrollments e ON c.course_id = e.course_id
WHERE e.student_id = 1;


--generate sql query to count number of students in each course
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
```

OUTPUT:

lab16.sql

```sql
-- generate  SQL create table stateme
--also define primary keys,foreign ke
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NUL
);

CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    credits INT NOT NULL
);

CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES st
    FOREIGN KEY (course_id) REFERENCES cou
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE students (
student_id INT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
email VARCHAR(100) UNIQUE NOT NULL
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE courses (
course_id INT PRIMARY KEY,
course_name VARCHAR(100) NOT NULL,
credits INT NOT NULL
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE enrollments (
student_id INT,
course_id INT,
PRIMARY KEY (student_id, course_id),
FOREIGN KEY (student_id) REFERENCES
students(student_id),
FOREIGN KEY (course_id) REFERENCES
courses(course_id)
);
```

lab16.sql

```sql

-- CREATE TABLE courses (
--     course_id INT PRIMARY KEY,
--     course_name VARCHAR(100) NOT NULL,
--     credits INT NOT NULL
-- );

-- CREATE TABLE enrollments (
--     student_id INT,
--     course_id INT,
--     PRIMARY KEY (student_id, course_id),
--     FOREIGN KEY (student_id) REFERENCES students(student_id),
--     FOREIGN KEY (course_id) REFERENCES courses(course_id)
-- );

--generate sql queries to insert sample data into the students, courses and
INSERT INTO students (student_id, first_name, last_name, email) VALUES
(1, 'John', 'Doe', 'john.doe@example.com'),
(2, 'Jane', 'Smith', 'jane.smith@example.com');

INSERT INTO courses (course_id, course_name, credits) VALUES
(101, 'Introduction to Computer Science', 3),
(102, 'Calculus I', 4);

INSERT INTO enrollments (student_id, course_id) VALUES
(1, 101),
(1, 102),
(2, 101);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO students (student_id, first_name, last_name, email) VALUES
(1, 'John', 'Doe', 'john.doe@example.com'),
(2, 'Jane', 'Smith', 'jane.smith@example.com');
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO courses (course_id, course_name, credits) VALUES
(101, 'Introduction to Computer Science', 3),
(102, 'Calculus I', 4);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO enrollments (student_id, course_id) VALUES
(1, 101),
(1, 102),
(2, 101);
```

lab16.sql

```sql
-- INSERT INTO enrollments (student_id, course_id) VALUES
-- (1, 101),
-- (1, 102),
-- (2, 101);

--generate sql query to insert a new student record
INSERT INTO students (student_id, first_name, last_name, email) VALUES
(3, 'Alice', 'Johnson', 'alice.johnson@example.com');
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO students (student_id, first_name, last_name, email) VALUES
(3, 'Alice', 'Johnson', 'alice.johnson@example.com');
```

lab16.sql

```sql
-- (3, 'Alice', 'Johnson', 'alice.johnson@example.com');

--generate sql query to fetch all courses enrolled by a student
SELECT c.course_name
FROM courses c
JOIN enrollments e ON c.course_id = e.course_id
WHERE e.student_id = 1;
```

SQL ▾ 1 / 1 1 - 2 of 2

course_name
Introduction to Computer Science
Calculus I

Labtest.py labtest2.py lab_15_1.py lab16.db lab16.sql SQLite

lab16.sql

```sql
--generate sql query to count number of students in each course
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
```

SQL ▾ 1 / 1 1 - 2 of 2

course_name	student_count
Calculus I	1
Introduction to Computer Science	2

EXPLANATION:

- Designed a relational database schema with Students, Courses, and Enrollments tables using primary and foreign keys.
- Implemented relationships using a junction table (Enrollments) to handle many-to-many mapping between students and courses.
- Performed CRUD operations including inserting student/course data and retrieving enrolled courses using JOIN queries.
- Used aggregate function (COUNT with GROUP BY) to calculate the number of students in each course.

Task 2 – Hospital Management Database

PROMPT:

create a schema for hospital management system

insert sample data into all tables

query to retrieve list of all appointments for a specific doctor

query to retrieve patient history by patient Id

query to count total patients treated by each doctor

CODE:

script.js | styles.css | index.html | L1.py | Labtest.py | labtest2.py | lab_15_1.py | lab16.db | lab16.sql

lab16.sql

```sql
--create a schema for hospital management system
CREATE TABLE patients (
    patient_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    date_of_birth DATE NOT NULL,
    gender VARCHAR(10) NOT NULL,
    phone VARCHAR(20),
    email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE doctors (
    doctor_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    specialty VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE appointments (
    appointment_id INT PRIMARY KEY,
    patient_id INT,
    doctor_id INT,
    appointment_date DATE NOT NULL,
    appointment_time TIME NOT NULL,
    status VARCHAR(20) DEFAULT 'Scheduled',
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
);


--insert sample data into all tables
INSERT INTO patients (patient_id, first_name, last_name, date_of_birth, gender, phone, email) VALUES
(1, 'John', 'Doe', '1980-01-01', 'Male', '123-456-7890', 'john.doe@example.com'),
(2, 'Jane', 'Smith', '1990-02-15', 'Female', '555-123-4567', 'jane.smith@example.com');
INSERT INTO doctors (doctor_id, first_name, last_name, specialty, phone, email) VALUES
```

script.js | styles.css | index.html | L1.py | Labtest.py | labtest2.py | lab_15_1.py | lab16.db | lab16.sql | index1.html

lab16.sql

```sql
--insert sample data into all tables
INSERT INTO patients (patient_id, first_name, last_name, date_of_birth, gender, phone, email) VALUES
(1, 'John', 'Doe', '1980-01-01', 'Male', '123-456-7890', 'john.doe@example.com'),
(2, 'Jane', 'Smith', '1990-02-15', 'Female', '555-123-4567', 'jane.smith@example.com');
INSERT INTO doctors (doctor_id, first_name, last_name, specialty, phone, email) VALUES
(1, 'Dr. Smith', 'Johnson', 'Cardiology', '987-654-3210', 'dr.smith@example.com'),
(2, 'Dr. Emily', 'Davis', 'Pediatrics', '555-987-6543', 'dr.davis@example.com');
INSERT INTO appointments (appointment_id, patient_id, doctor_id, appointment_date, appointment_time, status) VALUES
(1, 1, 1, '2024-07-01', '10:00:00', 'Scheduled'),
(2, 2, 2, '2024-07-02', '14:00:00', 'Scheduled');

--query to retrieve list of all appointments for a specific doctor
SELECT a.appointment_id, p.first_name AS patient_first_name, p.last_name AS patient_last_name, a.appointment_date, a.appointment_time, a.status
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
WHERE a.doctor_id = 1;



--query to retrieve patient history by patient Id
SELECT p.first_name AS patient_first_name, p.last_name AS patient_last_name, a.appointment_date, a.appointment_time, a.status, d.first_name AS docto
FROM patients p
JOIN appointments a ON p.patient_id = a.patient_id
JOIN doctors d ON a.doctor_id = d.doctor_id
WHERE p.patient_id = 1;



--query to count total patients treated by each doctor
SELECT d.first_name AS doctor_first_name, d.last_name AS doctor_last_name, COUNT(a
.appointment_id) AS total_patients
FROM doctors d
LEFT JOIN appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id;
```

OUTPUT:

Labtest.py | labtest2.py | lab_15_1.py | lab16.db | lab16.sql

lab16.sql

```sql
--create a schema for hospital management system
CREATE TABLE patients (
    patient_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    date_of_birth DATE NOT NULL,
    gender VARCHAR(10) NOT NULL,
    phone VARCHAR(20),
    email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE doctors (
    doctor_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    specialty VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE appointments (
    appointment_id INT PRIMARY KEY,
    patient_id INT,
    doctor_id INT,
    appointment_date DATE NOT NULL,
    appointment_time TIME NOT NULL,
    status VARCHAR(20) DEFAULT 'Scheduled',
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
);
```

SQLite

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE patients (
patient_id INT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
date_of_birth DATE NOT NULL,
gender VARCHAR(10) NOT NULL,
phone VARCHAR(20),
email VARCHAR(100) UNIQUE NOT NULL
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE doctors (
doctor_id INT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
specialty VARCHAR(100) NOT NULL,
phone VARCHAR(20),
email VARCHAR(100) UNIQUE NOT NULL
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE appointments (
appointment_id INT PRIMARY KEY,
patient_id INT,
doctor_id INT,
appointment_date DATE NOT NULL,
appointment_time TIME NOT NULL,
status VARCHAR(20) DEFAULT 'Scheduled',
FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
);
```

lab16.sql

```sql
CREATE TABLE appointments (
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
);


--insert sample data into all tables
INSERT INTO patients (patient_id, first_name, last_name, date_of_bi
(1, 'John', 'Doe', '1980-01-01', 'Male', '123-456-7890', 'john.doe@
(2, 'Jane', 'Smith', '1990-02-15', 'Female', '555-123-4567', 'jane.
INSERT INTO doctors (doctor_id, first_name, last_name, specialty, p
(1, 'Dr. Smith', 'Johnson', 'Cardiology', '987-654-3210', 'dr.smith
(2, 'Dr. Emily', 'Davis', 'Pediatrics', '555-987-6543', 'dr.davis@e
INSERT INTO appointments (appointment_id, patient_id, doctor_id, ap
(1, 1, 1, '2024-07-01', '10:00:00', 'Scheduled'),
(2, 2, 2, '2024-07-02', '14:00:00', 'Scheduled');
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO patients (patient_id, first_name, last_name, date_of_birth,
gender, phone, email) VALUES
(1, 'John', 'Doe', '1980-01-01', 'Male', '123-456-7890',
'john.doe@example.com'),
(2, 'Jane', 'Smith', '1990-02-15', 'Female', '555-123-4567',
'jane.smith@example.com');
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO doctors (doctor_id, first_name, last_name, specialty, phone,
email) VALUES
(1, 'Dr. Smith', 'Johnson', 'Cardiology', '987-654-3210',
'dr.smith@example.com'),
(2, 'Dr. Emily', 'Davis', 'Pediatrics', '555-987-6543',
'dr.davis@example.com');
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO appointments (appointment_id, patient_id, doctor_id,
appointment_date, appointment_time, status) VALUES
(1, 1, 1, '2024-07-01', '10:00:00', 'Scheduled'),
(2, 2, 2, '2024-07-02', '14:00:00', 'Scheduled');
```

```sql
(1, 1, 1, '2024-07-01', '10:00:00', 'Scheduled'),
(2, 2, 2, '2024-07-02', '14:00:00', 'Scheduled');

--query to retrieve list of all appointments for a specific doctor
SELECT a.appointment_id, p.first_name AS patient_first_name, p.last
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
WHERE a.doctor_id = 1;
```

SQL ▾ 1 / 1 1 - 1 of 1

appointment_id	patient_first_name	patient_last_name	appointment_date	appointm
1	John	Doe	2024-07-01	10:00:00

Labtest.py | labtest2.py | lab_15_1.py | lab16.db | lab16.sql | index | SQLite

lab16.sql

```sql
--query to retrieve patient history by patient Id
SELECT p.first_name AS patient_first_name, p.last_name AS patient_last_name
FROM patients p
JOIN appointments a ON p.patient_id = a.patient_id
JOIN doctors d ON a.doctor_id = d.doctor_id
WHERE p.patient_id = 1;
```

SQL ▾ 1 / 1 1 - 1 of 1

patient_first_name	patient_last_name	appointment_date	appointment_tim
John	Doe	2024-07-01	10:00:00

Labtest.py | labtest2.py | lab_15_1.py | lab16.db | lab16.sql | index | SQLite

lab16.sql

```sql
--query to count total patients treated by each doctor
SELECT d.first_name AS doctor_first_name, d.last_name AS doctor_last_name,
.appointment_id) AS total_patients
FROM doctors d
LEFT JOIN appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id;
```

SQL ▾ 1 / 1 1 - 2 of 2

doctor_first_name	doctor_last_name	total_patients
Dr. Smith	Johnson	1
Dr. Emily	Davis	1

EXPLANATION:

- Designed a hospital database schema with Patients, Doctors, and Appointments tables using primary and foreign keys.
- Established relationships by linking appointments with patients and doctors to manage scheduling and interactions.
- Inserted sample data and used JOIN queries to retrieve doctor appointments and patient history.
- Applied aggregate function (COUNT with GROUP BY) to calculate total patients treated by each doctor

Task 3 – Library Database

PROMPT:

generate a schema for a library management system that include tables books, members and loans

insert sample data into the tables

query to retrieve all books currently issued

query to find overdue books where loan date>30days

query to count number of books loaned by each member

CODE:

script.js | styles.css | index.html | L1.py | Labtest.py | labtest2.py | lab_15_1.py | lab16.db | lab16.

lab16.sql

```sql
--generate a schema for a library management system that include tables books, members and loans
CREATE TABLE books1 (
    book_id INT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    author VARCHAR(255) NOT NULL,
    published_date DATE,
    available_copies INT DEFAULT 0
);

CREATE TABLE members1 (
    member_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    join_date DATE
);

CREATE TABLE loans1 (
    loan_id INT PRIMARY KEY,
    book_id INT,
    member_id INT,
    loan_date DATE,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES books1(book_id),
    FOREIGN KEY (member_id) REFERENCES members(member_id)
);

--insert sample values into all tables
INSERT INTO books1 (book_id, title, author, published_date, available_copies) VALUES
(1, 'The Great Gatsby', 'F. Scott Fitzgerald', '1925-04-10', 3),
(2, 'To Kill a Mockingbird', 'Harper Lee', '1960-07-11', 2),
(3, '1984', 'George Orwell', '1949-06-08', 5);

INSERT INTO members1 (member_id, name, email, join_date) VALUES
(1, 'Alice Smith', 'alice.smith@example.com', '2023-01-15'),
(2, 'Bob Johnson', 'bob.johnson@example.com', '2023-02-20');

INSERT INTO loans1 (loan_id, book_id, member_id, loan_date, return_date) VALUES
```

script.js | styles.css | index.html | L1.py | Labtest.py | labtest2.py | lab_15_1.py | lab1

lab16.sql

```sql
(1, 'The Great Gatsby', 'F. Scott Fitzgerald', '1925-04-10', 3),
(2, 'To Kill a Mockingbird', 'Harper Lee', '1960-07-11', 2),
(3, '1984', 'George Orwell', '1949-06-08', 5);

INSERT INTO members1 (member_id, name, email, join_date) VALUES
(1, 'Alice Smith', 'alice.smith@example.com', '2023-01-15'),
(2, 'Bob Johnson', 'bob.johnson@example.com', '2023-02-20');

INSERT INTO loans1 (loan_id, book_id, member_id, loan_date, return_date) VALUES
(1, 1, 1, '2023-03-01', NULL),
(2, 2, 2, '2023-03-05', '2023-03-20'),
(3, 3, 1, '2023-03-10', NULL);


--query to retrieve all books currently issued
SELECT b.title, m.name, l.loan_date
FROM loans1 l
JOIN books1 b ON l.book_id = b.book_id
JOIN members1 m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;


--query to find overdue books where loan date>30days
SELECT b.title, m.name, l.loan_date
FROM loans1 l
JOIN books1 b ON l.book_id = b.book_id
JOIN members1 m ON l.member_id = m.member_id
WHERE l.return_date IS NULL
AND l.loan_date < DATE('now', '-30 days');

--query to count number of books loaned by each member
SELECT m.name, COUNT(l.book_id) AS books_loaned
FROM loans1 l
JOIN members1 m ON l.member_id = m.member_id
GROUP BY m.name;
```

OUTPUT:

lab16.sql

```sql
--generate a schema for a library management system that include ta
CREATE TABLE books1 (
    book_id INT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    author VARCHAR(255) NOT NULL,
    published_date DATE,
    available_copies INT DEFAULT 0
);

CREATE TABLE members1 (
    member_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    join_date DATE
);

CREATE TABLE loans1 (
    loan_id INT PRIMARY KEY,
    book_id INT,
    member_id INT,
    loan_date DATE,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES books1(book_id),
    FOREIGN KEY (member_id) REFERENCES members(member_id)
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE books1 (
book_id INT PRIMARY KEY,
title VARCHAR(255) NOT NULL,
author VARCHAR(255) NOT NULL,
published_date DATE,
available_copies INT DEFAULT 0
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE members1 (
member_id INT PRIMARY KEY,
name VARCHAR(255) NOT NULL,
email VARCHAR(255) UNIQUE NOT NULL,
join_date DATE
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE loans1 (
loan_id INT PRIMARY KEY,
book_id INT,
member_id INT,
loan_date DATE,
return_date DATE,
FOREIGN KEY (book_id) REFERENCES books1(book_id),
FOREIGN KEY (member_id) REFERENCES members(member_id)
);
```

lab16.sql

```sql
CREATE TABLE members1 (
    member_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    join_date DATE
);

CREATE TABLE loans1 (
    loan_id INT PRIMARY KEY,
    book_id INT,
    member_id INT,
    loan_date DATE,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES books1(book_id),
    FOREIGN KEY (member_id) REFERENCES members(member_id)
);

--insert sample values into all tables
INSERT INTO books1 (book_id, title, author, published_date, availab
(1, 'The Great Gatsby', 'F. Scott Fitzgerald', '1925-04-10', 3),
(2, 'To Kill a Mockingbird', 'Harper Lee', '1960-07-11', 2),
(3, '1984', 'George Orwell', '1949-06-08', 5);

INSERT INTO members1 (member_id, name, email, join_date) VALUES
(1, 'Alice Smith', 'alice.smith@example.com', '2023-01-15'),
(2, 'Bob Johnson', 'bob.johnson@example.com', '2023-02-20');

INSERT INTO loans1 (loan_id, book_id, member_id, loan_date, return_date) Va
(1, 1, 1, '2023-03-01', NULL),
(2, 2, 2, '2023-03-05', '2023-03-20'),
(3, 3, 1, '2023-03-10', NULL);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO books1 (book_id, title, author, published_date, available_co
VALUES
(1, 'The Great Gatsby', 'F. Scott Fitzgerald', '1925-04-10', 3),
(2, 'To Kill a Mockingbird', 'Harper Lee', '1960-07-11', 2),
(3, '1984', 'George Orwell', '1949-06-08', 5);
```

lab16.sql

```sql
--insert sample values into all tables
INSERT INTO books1 (book_id, title, author, published_date, availab
(1, 'The Great Gatsby', 'F. Scott Fitzgerald', '1925-04-10', 3),
(2, 'To Kill a Mockingbird', 'Harper Lee', '1960-07-11', 2),
(3, '1984', 'George Orwell', '1949-06-08', 5);

INSERT INTO members1 (member_id, name, email, join_date) VALUES
(1, 'Alice Smith', 'alice.smith@example.com', '2023-01-15'),
(2, 'Bob Johnson', 'bob.johnson@example.com', '2023-02-20');

INSERT INTO loans1 (loan_id, book_id, member_id, loan_date, return_date) Va
(1, 1, 1, '2023-03-01', NULL),
(2, 2, 2, '2023-03-05', '2023-03-20'),
(3, 3, 1, '2023-03-10', NULL);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO members1 (member_id, name, email, join_date) VALUES
(1, 'Alice Smith', 'alice.smith@example.com', '2023-01-15'),
(2, 'Bob Johnson', 'bob.johnson@example.com', '2023-02-20');
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO loans1 (loan_id, book_id, member_id, loan_date, return_date)
VALUES
(1, 1, 1, '2023-03-01', NULL),
(2, 2, 2, '2023-03-05', '2023-03-20'),
(3, 3, 1, '2023-03-10', NULL);
```

lab16.sql

```sql
(1, 1, 1, '2023-03-01', NULL),
(2, 2, 2, '2023-03-05', '2023-03-20'),
(3, 3, 1, '2023-03-10', NULL);


--query to retrieve all books currently issued
SELECT b.title, m.name, l.loan_date
FROM loans1 l
JOIN books1 b ON l.book_id = b.book_id
JOIN members1 m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
```

SQL ▾ 1 / 1 1 - 2 of 2

```sql
SELECT b.title, m.name, l.loan_date
FROM loans1 l
JOIN books1 b ON l.book_id = b.book_id
JOIN members1 m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
```

title	name	loan_date
The Great Gatsby	Alice Smith	2023-03-01
1984	Alice Smith	2023-03-10

lab16.sql

```sql
--query to find overdue books where loan date>30days
SELECT b.title, m.name, l.loan_date
FROM loans1 l
JOIN books1 b ON l.book_id = b.book_id
JOIN members1 m ON l.member_id = m.member_id
WHERE l.return_date IS NULL
AND l.loan_date < DATE('now', '-30 days');
```

SQL ▾ 1 / 1 1 - 2 of 2

title	name	loan_date
The Great Gatsby	Alice Smith	2023-03-01
1984	Alice Smith	2023-03-10

labtest.py labtest2.py lab_15_1.py lab16.db lab16.sql index1.html SQLite

lab16.sql

```sql
--query to count number of books loaned by each member
SELECT m.name, COUNT(l.book_id) AS books_loaned
FROM loans1 l
JOIN members1 m ON l.member_id = m.member_id
GROUP BY m.name;
```

SQL ▾ 1 / 1 1 - 2 of 2

name	books_loaned
Alice Smith	2
Bob Johnson	1

EXPLANATION:

- Designed a library database with Books, Members, and Loans tables using primary and foreign keys to manage relationships.
- Inserted sample data and used JOIN queries to retrieve issued books and track member borrowing activity.
- Identified overdue books using date conditions (DATE('now', '-30 days')) for time-based filtering.
- Applied aggregate function (COUNT with GROUP BY) to calculate the number of books borrowed by each member.

Task 4 – Real-Time Application: Online Shopping Database

PROMPT:

--generate a scheme for real time application online shopping database that include tables

-- users products orders and orderdetails in sqlite syntax

insert sample data into all tables

query to retrieve all orders by a user in sqlite synta

query to find the most purchased product

-query to calculate total revenue in a given month using sqlite syntax

CODE:

```sql
--generate a scheme for real time application online shopping database that include tables
-- users products orders and orderdetails in sqlite syntax
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL UNIQUE,
    email TEXT NOT NULL UNIQUE,
    password TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    description TEXT,
    price REAL NOT NULL,
    stock INTEGER NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    total_amount REAL NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE order_details (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL,
    price REAL NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES orders(id),
    FOREIGN KEY (product_id) REFERENCES products(id)
);
```

script.js | styles.css | index.html | L1.py | Labtest.py | labtest2.py | lab_15_1.py | lab16.db | la

lab16.sql

```sql


--insert sample data into all tables
INSERT INTO users (username, email, password) VALUES
('john_doe', 'john.doe@example.com', 'hashed_password_1'),
('jane_smith', 'jane.smith@example.com', 'hashed_password_2');

INSERT INTO products (name, description, price, stock) VALUES
('Laptop', 'A high-performance laptop', 999.99, 10),
('Smartphone', 'A latest model smartphone', 499.99, 20);

INSERT INTO orders (user_id, total_amount) VALUES
(1, 1499.98),
(2, 499.99);

INSERT INTO order_details (order_id, product_id, quantity, price) VALUES
(1, 1, 1, 999.99),
(1, 2, 1, 499.99),
(2, 2, 1, 499.99);

--query to retrieve all orders by a user in sqlite syntax
select o.id, o.total_amount, o.created_at
from orders o
join users u on o.user_id = u.id
where u.username = 'john_doe';




--query to find the most purchased product
SELECT p.name, SUM(od.quantity) AS total_quantity
FROM products p
JOIN order_details od ON p.id = od.product_id
GROUP BY p.id
ORDER BY total_quantity DESC
LIMIT 1;
```

lab16.sql

```sql
join users u on o.user_id = u.id
where u.username = 'john_doe';




--query to find the most purchased product
SELECT p.name, SUM(od.quantity) AS total_quantity
FROM products p
JOIN order_details od ON p.id = od.product_id
GROUP BY p.id
ORDER BY total_quantity DESC
LIMIT 1;



-- --query to calculate total revenue in a given month using sqlite syntax
SELECT strftime('%Y-%m', created_at) AS month, SUM(total_amount) AS total_revenue
FROM orders
WHERE created_at >= '2024-01-01' AND created_at < '2024-02-01'
GROUP BY month;
```

OUTPUT:

lab16.sql

```sql
--generate a scheme for real time application online shopping datab
-- users products orders and orderdetails in sqlite syntax
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL UNIQUE,
    email TEXT NOT NULL UNIQUE,
    password TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    description TEXT,
    price REAL NOT NULL,
    stock INTEGER NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    total_amount REAL NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE order_details (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL,
    price REAL NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES orders(id),
    FOREIGN KEY (product_id) REFERENCES products(id)
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE users (
id INTEGER PRIMARY KEY AUTOINCREMENT,
username TEXT NOT NULL UNIQUE,
email TEXT NOT NULL UNIQUE,
password TEXT NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE products (
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT NOT NULL,
description TEXT,
price REAL NOT NULL,
stock INTEGER NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE orders (
id INTEGER PRIMARY KEY AUTOINCREMENT,
user_id INTEGER NOT NULL,
total_amount REAL NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(id)
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE order_details (
id INTEGER PRIMARY KEY AUTOINCREMENT,
order_id INTEGER NOT NULL,
product_id INTEGER NOT NULL,
```

lab16.sql

```sql
CREATE TABLE order_details (
    FOREIGN KEY (product_id) REFERENCES products(id)
);


--insert sample data into all tables
INSERT INTO users (username, email, password) VALUES
('john_doe', 'john.doe@example.com', 'hashed_password_1'),
('jane_smith', 'jane.smith@example.com', 'hashed_password_2');

INSERT INTO products (name, description, price, stock) VALUES
('Laptop', 'A high-performance laptop', 999.99, 10),
('Smartphone', 'A latest model smartphone', 499.99, 20);

INSERT INTO orders (user_id, total_amount) VALUES
(1, 1499.98),
(2, 499.99);

INSERT INTO order_details (order_id, product_id, quantity, price) VALUES
(1, 1, 1, 999.99),
(1, 2, 1, 499.99),
(2, 2, 1, 499.99);




--query to retrievea all orders by a user
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO users (username, email, password) VALUES
('john_doe', 'john.doe@example.com', 'hashed_password_1'),
('jane_smith', 'jane.smith@example.com', 'hashed_password_2');
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO products (name, description, price, stock) VALUES
('Laptop', 'A high-performance laptop', 999.99, 10),
('Smartphone', 'A latest model smartphone', 499.99, 20);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO orders (user_id, total_amount) VALUES
(1, 1499.98),
(2, 499.99);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO order_details (order_id, product_id, quantity, price) VALUES
(1, 1, 1, 999.99),
(1, 2, 1, 499.99),
(2, 2, 1, 499.99);
```

lab16.sql

```sql
-- (1, 1, 1, 999.99),
-- (1, 2, 1, 499.99),
-- (2, 2, 1, 499.99);

--query to retrieve all orders by a user in sqlite syntax
select o.id, o.total_amount, o.created_at
from orders o
join users u on o.user_id = u.id
where u.username = 'john_doe';
```

SQL ▾ 1 / 1 1 - 1 of 1

id	total_amount	created_at
1	1499.98	2026-03-24 18:01:50

lab16.sql

```sql
join users u on o.user_id = u.id
where u.username = 'john_doe';



--query to find the most purchased product
SELECT p.name, SUM(od.quantity) AS total_quantity
FROM products p
JOIN order_details od ON p.id = od.product_id
GROUP BY p.id
ORDER BY total_quantity DESC
LIMIT 1;
```

SQL ▾ 1 / 1 1 - 1 of 1

name	total_quantity
Smartphone	2

EXPLANATION:

- Designed an online shopping database with Users, Products, Orders, and Order Details tables using primary and foreign keys to maintain relationships.
- Inserted sample data and used JOIN queries to retrieve user orders and identify the most purchased product.
- Applied aggregate functions like SUM and GROUP BY to calculate total quantity and revenue.
- Used SQLite date function (strftime) to filter and compute monthly revenue efficiently.

Task 5 – University Examination Database

PROMPT:

design a database schema for a university examination system with tables for students , subjects ,exams and results also define primary keys,foreign keys and referential integrity constraints

insert sample data into the tables

query to insert examination results for a student

query to Retrieve subject-wise marks for a student

query to Calculate average marks for each subject

CODE:

lab16.sql

```sql
--design a database schema for a university examination system
--with tables for students , subjects ,exams and results
--also define primary keys,foreign keys and referential integrity constrsints.
CREATE TABLE students1 (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL
);
Create TABLE subjects (
    subject_id INT PRIMARY KEY,
    subject_name VARCHAR(100) NOT NULL
);
CREATE TABLE exams (
    exam_id INT PRIMARY KEY,
    subject_id INT,
    exam_date DATE NOT NULL,
    FOREIGN KEY (subject_id) REFERENCES subjects(subject_id) ON DELETE CASCADE
);
CREATE TABLE results (
    result_id INT PRIMARY KEY,
    student_id INT,
    exam_id INT,
    score DECIMAL(5,2) NOT NULL,
    FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE CASCADE,
    FOREIGN KEY (exam_id) REFERENCES exams(exam_id) ON DELETE CASCADE
);

--insert sample data into the tables
INSERT INTO students1 (student_id, first_name, last_name, email) VALUES
(1, 'John', 'Doe', 'john.doe@university.edu'),
(2, 'Jane', 'Smith', 'jane.smith@university.edu');

INSERT INTO subjects (subject_id, subject_name) VALUES
(1, 'Mathematics'),
(2, 'Physics');
```

lab16.sql

```sql
INSERT INTO subjects (subject_id, subject_name) VALUES
(1, 'Mathematics'),
(2, 'Physics');

INSERT INTO exams (exam_id, subject_id, exam_date) VALUES
(1, 1, '2024-12-01'),
(2, 2, '2024-12-02');

INSERT INTO results (result_id, student_id, exam_id, score) VALUES
(1, 1, 1, 85.50),
(2, 1, 2, 90.00),
(3, 2, 1, 78.00),
(4, 2, 2, 88.50);


--query to insert examination results for a student
INSERT INTO results (result_id, student_id, exam_id, score) VALUES
(5, 1, 1, 92.00);


--query to Retrieve subject-wise marks for a student
SELECT s.first_name, s.last_name, sub.subject_name, r.score
FROM students1 s
JOIN results r ON s.student_id = r.student_id
JOIN exams e ON r.exam_id = e.exam_id
JOIN subjects sub ON e.subject_id = sub.subject_id
WHERE s.student_id = 1;

--query to Calculate average marks for each subject
SELECT sub.subject_name, AVG(r.score) AS average_score
FROM subjects sub
JOIN exams e ON sub.subject_id = e.subject_id
JOIN results r ON e.exam_id = r.exam_id
GROUP BY sub.subject_name;

```

OUTPUT:

Labtest.py | labtest2.py | lab_15_1.py | lab16.db | lab16.sql

lab16.sql

```sql
--design a database schema for a university examination system
--with tables for students , subjects ,exams and results
--also define primary keys,foreign keys and referential integrity c
CREATE TABLE students1 (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL
);
Create TABLE subjects (
    subject_id INT PRIMARY KEY,
    subject_name VARCHAR(100) NOT NULL
);
CREATE TABLE exams (
    exam_id INT PRIMARY KEY,
    subject_id INT,
    exam_date DATE NOT NULL,
    FOREIGN KEY (subject_id) REFERENCES subjects(subject_id) ON DEL
);
CREATE TABLE results (
    result_id INT PRIMARY KEY,
    student_id INT,
    exam_id INT,
    score DECIMAL(5,2) NOT NULL,
    FOREIGN KEY (student_id) REFERENCES students(student_id) ON DEL
    FOREIGN KEY (exam_id) REFERENCES exams(exam_id) ON DELETE CASCA
);
```

SQLite

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE students1 (
student_id INT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
email VARCHAR(100) UNIQUE NOT NULL
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
Create TABLE subjects (
subject_id INT PRIMARY KEY,
subject_name VARCHAR(100) NOT NULL
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE exams (
exam_id INT PRIMARY KEY,
subject_id INT,
exam_date DATE NOT NULL,
FOREIGN KEY (subject_id) REFERENCES subjects(subject_id) ON DELETE CASCADE
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE results (
result_id INT PRIMARY KEY,
student_id INT,
exam_id INT,
score DECIMAL(5,2) NOT NULL,
FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE CASCADE,
FOREIGN KEY (exam_id) REFERENCES exams(exam_id) ON DELETE CASCADE
);
```

Labtest.py | labtest2.py | lab_15_1.py | lab16.db | lab16.sql

lab16.sql

```sql
--insert sample data into the tables
INSERT INTO students1 (student_id, first_name, last_name, email) VA
(1, 'John', 'Doe', 'john.doe@university.edu'),
(2, 'Jane', 'Smith', 'jane.smith@university.edu');

INSERT INTO subjects (subject_id, subject_name) VALUES
(1, 'Mathematics'),
(2, 'Physics');

INSERT INTO exams (exam_id, subject_id, exam_date) VALUES
(1, 1, '2024-12-01'),
(2, 2, '2024-12-02');

INSERT INTO results (result_id, student_id, exam_id, score) VALUES
(1, 1, 1, 85.50),
(2, 1, 2, 90.00),
(3, 2, 1, 78.00),
(4, 2, 2, 88.50);
```

SQLite

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO students1 (student_id, first_name, last_name, email) VALUES
(1, 'John', 'Doe', 'john.doe@university.edu'),
(2, 'Jane', 'Smith', 'jane.smith@university.edu');
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO subjects (subject_id, subject_name) VALUES
(1, 'Mathematics'),
(2, 'Physics');
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO exams (exam_id, subject_id, exam_date) VALUES
(1, 1, '2024-12-01'),
(2, 2, '2024-12-02');
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO results (result_id, student_id, exam_id, score) VALUES
(1, 1, 1, 85.50),
(2, 1, 2, 90.00),
(3, 2, 1, 78.00),
(4, 2, 2, 88.50);
```

Labtest.py | labtest2.py | lab_15_1.py | lab16.db | lab16.sql

lab16.sql

```sql
(3, 2, 1, 78.00),
(4, 2, 2, 88.50);


--query to insert examination results for a student
INSERT INTO results (result_id, student_id, exam_id, score) VALUES
(5, 1, 1, 92.00);
```

SQLite

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO results (result_id, student_id, exam_id, score) VALUES
(5, 1, 1, 92.00);
```

Labtest.py | labtest2.py | lab_15_1.py | lab16.db | lab16.sql

lab16.sql

```sql
(4, 2, 2, 88.50);


--query to insert examination results for a student
INSERT INTO results (result_id, student_id, exam_id, score) VALUES
(5, 1, 1, 92.00);


--query to Retrieve subject-wise marks for a student
SELECT s.first_name, s.last_name, sub.subject_name, r.score
FROM students1 s
JOIN results r ON s.student_id = r.student_id
JOIN exams e ON r.exam_id = e.exam_id
JOIN subjects sub ON e.subject_id = sub.subject_id
WHERE s.student_id = 1;
```

SQLite

SQL ▾ 1 / 1 1 - 3 of 3

```sql
SELECT s.first_name, s.last_name, sub.subject_name, r.score
FROM students1 s
JOIN results r ON s.student_id = r.student_id
JOIN exams e ON r.exam_id = e.exam_id
JOIN subjects sub ON e.subject_id = sub.subject_id
WHERE s.student_id = 1;
```

first_name	last_name	subject_name	score
John	Doe	Mathematics	85.5
John	Doe	Physics	90
John	Doe	Mathematics	92

lab16.sql

```sql
--query to Calculate average marks for each subject
SELECT sub.subject_name, AVG(r.score) AS average_score
FROM subjects sub
JOIN exams e ON sub.subject_id = e.subject_id
JOIN results r ON e.exam_id = r.exam_id
GROUP BY sub.subject_name;
```

SQL ▾ 1 / 1 1 - 2 of 2

```sql
SELECT sub.subject_name, AVG(r.score) AS average_score
FROM subjects sub
JOIN exams e ON sub.subject_id = e.subject_id
JOIN results r ON e.exam_id = r.exam_id
GROUP BY sub.subject_name;
```

subject_name	average_score
Mathematics	85.1666666666667
Physics	89.25

EXPLANATION:

- Designed a university examination database with Students, Subjects, Exams, and Results tables using primary and foreign keys to maintain relationships.
- Ensured referential integrity by linking results with students and exams, and exams with subjects.
- Inserted sample data and used JOIN queries to retrieve subject-wise marks for a student.
- Applied aggregate function (AVG with GROUP BY) to calculate average marks for each subject.